

Learnmate (AI StudyBuddy)

System Architecture, API Specification & Database Design Documentation

Environment: Node.js / Express / MongoDB

AI Integration: Google Generative AI (gemini-3.7-flash)

Document Version: 1.0.0

1. Introduction

Learnmate (internally configured as **AI StudyBuddy**) is a robust Node.js/Express web application backend designed to streamline educational content delivery and study material management. The platform integrates modern generative artificial intelligence to empower students and administrators by automatically transforming unstructured educational assets (such as raw text files, Markdown documents, and PDF manuals) into structured learning resources including summaries, flashcards, interactive quizzes, and personalized study plans.

2. Literature Survey

Modern digital learning platforms face challenges in personalizing content at scale and automating assessment generation from continuous reading materials. Conventional Learning Management Systems (LMS) typically rely on manual quiz creation and static file hosting. **Learnmate** bridges this gap by incorporating cutting-edge Generative AI directly into document processing pipelines. By utilizing Google's `gemini-3.7-flash` model alongside Role-Based Access Control (RBAC) and structured MongoDB persistence, Learnmate establishes an automated end-to-end framework for interactive study material processing.

3. System Requirements

Hardware Requirements

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher.
- **RAM:** 4 GB minimum (8 GB recommended for concurrent AI requests).
- **Storage:** 500 MB minimum available disk space for source files and local upload directory storage.

Software Requirements

- **Operating System:** Cross-platform (Windows 10/11, macOS, Linux).
- **Runtime Environment:** Node.js (v16.x or higher).
- **Database Engine:** MongoDB (Local Instance or MongoDB Atlas).
- **API Client / Testing Tool:** Postman.
- **Development Environment:** Visual Studio Code.

4. Technology Stack

LAYER / COMPONENT	TECHNOLOGY / LIBRARY	PURPOSE & FUNCTIONALITY
Backend Framework	Node.js & Express.js	RESTful API routing, middleware execution, and async server handling.
Database Layer	MongoDB & Mongoose ORM	NoSQL document persistence, schema validation, and object modeling.
Security & Auth	JSON Web Tokens (JWT) & bcryptjs	Stateless authentication via Bearer tokens and salted password hashing (salt factor 10).
File Ingestion	Multer Middleware	Multipart form-data parsing, file type validation, and disk storage management.
Generative AI	@google/generative-ai	Direct SDK integration with the <code>gemini-3.7-flash</code> language model.
Configuration	dotenv & cors	Environment variable isolation and cross-origin resource sharing control.

5. System Analysis

Existing vs. Proposed System

Existing Systems: Manual note synthesis, standard non-interactive document repositories, and independent third-party quiz generators requiring manual content re-formatting.

Proposed System (Learnmate): A unified backend solution capable of accepting document uploads (`.txt`, `.md`, `.pdf`), storing source files, parsing content, and programmatically generating summaries, structured flashcard arrays, option-based multiple-choice quizzes, and custom study itineraries.

Feasibility Analysis

- **Technical Feasibility:** Employs non-blocking asynchronous I/O with `express-async-errors` and robust exponential retry backoff logic (handling 503/429 status codes) for seamless API resilience.
- **Operational Feasibility:** Clean, modular separation into controllers, routes, models, and utility modules ensuring maintainability.
- **Economic Feasibility:** Leverages open-source backend stacks and scalable cloud APIs, minimizing upfront infrastructure overhead.

6. System Design

Project Directory Structure

```
Learnmate/
├── .env
├── .postman/
├── index.js
├── package.json
├── package-lock.json
├── src/
│   ├── controllers/
│   │   ├── adminController.js
│   │   ├── authController.js
│   │   └── materialController.js
│   ├── middleware/
│   │   ├── auth.js
│   │   └── upload.js
│   ├── models/
│   │   ├── Material.js
│   │   └── User.js
│   ├── routes/
│   │   ├── admin.js
│   │   ├── auth.js
│   │   └── materials.js
│   └── utils/
│       ├── db.js
│       ├── gemini.js
│       └── tokens.js
```

System Workflow & Execution Pipeline

1. **Client Request:** User sends an HTTP request (e.g., login, file upload) to defined API routes.
2. **Middleware Execution:** Request passes through CORS, JSON body parser, JWT authentication checks (`protect`), and file validation filters (`upload`).
3. **Controller Operations:** Validated requests invoke corresponding controller logic to query MongoDB or interface with AI services.
4. **AI Retry Loop:** AI tasks execute via `askGemini()` with a 3-tier exponential backoff algorithm (`5000 * Math.pow(2, attempt)`) to handle temporary API availability issues.

7. Module Description

Module 1: Authentication & Authorization

Provides endpoint access security. Passwords are automatically hashed prior to insertion using Mongoose pre-save middleware. JWT tokens are verified on protected routes, and role parameters enforce strict access limits between standard students and platform administrators.

Module 2: File Upload & Filtering

Configured via Multer disk storage. Incoming files receive a randomized unique timestamp filename stored in the local `uploads/` directory. Allowed file formats are strictly limited to `.txt`, `.md`, and `.pdf`, with a strict maximum payload size of 5 MB.

Module 3: Study Material & Gemini AI Generation

Core domain logic responsible for processing learning materials. Integrates the Google Generative AI SDK using the `gemini-3.7-flash` model to extract key information and generate interactive learning assets directly linked to user profiles.

Module 4: Administrative Management

Enables privileged users (role: `admin`) to review, monitor, and remove user accounts or platform-wide uploaded study content.

8. Database Design

Schema 1: User Model (`src/models/User.js`)

- `name` (String, Required, Trimmed): User's full name.
- `email` (String, Required, Unique, Lowercase): User login email.
- `password` (String, Required): Salted password hash generated via `bcryptjs`.
- `role` (String, Enum: `["student", "admin"]`, Default: `"student"`): RBAC user privilege level.
- `timestamps`: Automatic `createdAt` and `updatedAt` tracking.
- **Schema Method:** `matchPassword(enteredPassword)` — performs asynchronous password hash comparison.

Schema 2: Material Model (`src/models/Material.js`)

- `user` (ObjectId, Ref: `"User"`, Required): Reference to material owner.
- `title` (String, Required): Title of the study material.
- `content` (String, Required): Raw text content extracted from source document.
- `filename` (String): Original name of uploaded document.
- `summary` (String): AI-generated conceptual summary.
- `flashcards`: Array of Objects `[{ question: String, answer: String }]`.
- `quiz`: Array of Objects `[{ question: String, options: [String], answer: String }]`.
- `studyPlan` (String): Structured study roadmap generated by AI.
- `timestamps`: Automatic creation and modification timestamps.

9. API Design and Implementation

Server entry point initialized via `index.js` running on `PORT 5000` by default, configured with health-check endpoints and global asynchronous error boundaries.

METHOD	ENDPOINT PATH	AUTH REQUIRED	MIDDLEWARE / ROLES	DESCRIPTION
GET	/	No	Public	Server status check. Returns API status message.
POST	/api/auth/register	No	Public	Registers new student or admin account.
POST	/api/auth/login	No	Public	Authenticates credentials & returns signed JWT token.
POST	/api/materials	Yes	<code>protect</code> , <code>upload.single()</code>	Uploads file (.txt, .md, .pdf) & creates material.
GET	/api/materials	Yes	<code>protect</code>	Retrieves all study materials belonging to active user.
ALL	/api/admin/*	Yes	<code>protect</code> , <code>adminOnly</code>	Privileged routes for site administration.

10. Testing and Results

- **Database Verification:** Verified DB initialization via `connectDB()` in `db.js` logging MongoDB connection host successfully.
- **Authentication Flow:** Verified registration, JWT issuance, password verification via `bcryptjs`, and route protection rejecting unauthorized tokens with `401 Not authenticated` or `403 Admins only` responses.
- **File Upload Control:** Confirmed rejection of non-permitted document formats (.exe, .png) while validating successful disk storage for `.pdf`, `.txt`, and `.md` payloads.
- **API Integrity Testing:** Executed end-to-end route validation utilizing dedicated Postman collection scripts (`.postman/`) ensuring accurate standard response payloads and error handling middleware execution.

11. Conclusion

The **Learnmate** backend architecture provides an efficient, scalable, and secure API infrastructure for next-generation educational applications. By combining structured relational data models in MongoDB, secure JWT/bcrypt authentication, strict file upload constraints, and automated generative AI processing via Google's Gemini platform, Learnmate establishes a strong foundational service layer for modern digital learning systems.